WEEK 3 · LESSON 18 OF 24 · HANDOUT

Week 3 Project: Build a Trustworthy Loan Model

Study time: **about 90 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=85dtiMz9tSo — open the lesson online to watch inside the course.

🎓 **Go deeper:** Data School: Selecting the best model using cross-validation — https://www.youtube.com/watch?v=6dbrR-WymjI

🎓 **Go deeper:** freeCodeCamp: Machine Learning with Python and scikit-learn — full course — https://www.youtube.com/watch?v=hDKCxebp88A

Your goal in this lesson: Deliver a complete, honest classification project: compare and tune models with cross-validation, choose a threshold from business costs, and report results from a single final test.

The brief

A Rwandan savings and credit cooperative (SACCO) wants to approve mobile micro-loans automatically. The board asks three questions:

1. Which model should we use, and how good is it really?
2. At what risk level should we refuse a loan?
3. What are the risks of using this model?

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

Step 1 — Lock the test set away

```
from sklearn.model_selection import train_test_split

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=0.25, random_state=7, stratify=y)
print(len(X_dev), "rows to develop with,", len(X_test), "rows locked away")
```

Everything until Step 5 uses **X_dev** only.

Step 2 — Compare candidates with cross-validation

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

cv = StratifiedKFold(5, shuffle=True, random_state=7)
candidates = {
    "Baseline": DummyClassifier(strategy="prior"),
    "Logistic regression": make_pipeline(StandardScaler(), LogisticRegression()),
    "Random forest": RandomForestClassifier(300, min_samples_leaf=5, random_state=7,
n_jobs=-1),
    "Gradient boosting": GradientBoostingClassifier(learning_rate=0.05, n_estimators=200,
                                                    max_depth=2, random_state=7),
}
for name, m in candidates.items():
    s = cross_val_score(m, X_dev, y_dev, cv=cv, scoring="roc_auc")
    print(f"{name:20s} AUC {s.mean():.3f} +/- {s.std():.3f}")
```

The baseline has an AUC of **0.5** (it gives everyone the same score). The three real models should all be around **0.85–0.87**.

Step 3 — Tune the leader

Tune the strongest simple model (or your favourite) with `GridSearchCV`:

```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(make_pipeline(StandardScaler(), LogisticRegression()),
                      {"logisticregression__C": [0.01, 0.1, 1, 10, 100]},
                      cv=cv, scoring="roc_auc")
search.fit(X_dev, y_dev)
print(search.best_params_, round(search.best_score_, 3))
model = search.best_estimator_
```

Step 4 — Choose the threshold on development data

Use cross-validated probabilities, so each development row is scored by a model that did not train on it:

```
import numpy as np
from sklearn.model_selection import cross_val_predict

dev_proba = cross_val_predict(model, X_dev, y_dev, cv=cv, method="predict_proba")[:, 1]
amount, actual = X_dev["loan_rwf"].to_numpy(), y_dev.to_numpy()

def profit(proba, amount, actual, t, fee=0.15, loss=0.60):
    ok = proba < t
    return fee * amount[ok & (actual == 0)].sum() - loss * amount[ok & (actual == 1)].sum()

thresholds = np.arange(0.05, 0.96, 0.05)
profits = [profit(dev_proba, amount, actual, t) for t in thresholds]
best_t = round(float(thresholds[int(np.argmax(profits))]), 2)
print("Best threshold on development data:", best_t)
```

Step 5 — Open the test set once

```
from sklearn.metrics import roc_auc_score, precision_score, recall_score

test_proba = model.predict_proba(X_test)[:, 1]
refuse = (test_proba >= best_t).astype(int)
print("Test AUC:", round(roc_auc_score(y_test, test_proba), 3))
print("Recall (defaulters refused):", round(recall_score(y_test, refuse), 3))
print("Precision (refused who would default):", round(precision_score(y_test, refuse), 3))
print("Approved:", int((refuse == 0).sum()), "of", len(refuse))
print("Profit: %.2f million RWF" % (profit(test_proba, X_test["loan_rwf"].to_numpy(),
                                           y_test.to_numpy(), best_t) / 1e6))
print("Profit if everyone is approved: %.2f million RWF" % (profit(test_proba,
X_test["loan_rwf"].to_numpy(),
                                                                  y_test.to_numpy(), 1.01) /
1e6))
```

Step 6 — Board report (one page)

1. **Recommendation** — the model, the threshold, and the expected profit compared with approving everyone.
2. **How good** — cross-validated AUC with its ± spread, and the single test AUC.
3. **What it means for customers** — how many good borrowers are refused per 100 applicants.
4. **What drives risk** — the logistic coefficients, in plain words.
5. **Risks and safeguards** — data drift (incomes change), fairness (check results by group, Week 4), a human appeal process for refused customers, and retraining every few months.

Marking guide (self-check)

Area	Excellent work
Honesty	Test set opened once, after all choices
Method	Cross-validation and tuning inside pipelines
Business sense	Threshold chosen from costs on development data
Customers	Effect on good borrowers stated clearly
Responsibility	Drift, fairness and appeals discussed

Remember this: A trustworthy model comes from an honest process: develop with cross-validation, decide the threshold from real costs, and open the test set only once.

Name: _____ Date: _____

Activity

Task 1

Complete Steps 1–6. Then repeat Step 4 with a 25% fee and a 50% loss and explain to the board how the recommended threshold would change.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why is the test set split off before anything else?
   A. To scale it
   B. So no choice is influenced by it and the final score stays honest
   C. To train faster
   D. Because it is bigger
2. What AUC does the DummyClassifier baseline get?
   A. 1.0
   B. 0.5
   C. 0.79
   D. 0.0
3. Why use cross_val_predict to choose the threshold?
   A. It removes the test set
   B. Each development row gets a probability from a model that did not train on it
   C. It is faster
   D. It changes the labels
4. How was the threshold chosen?
   A. The value that maximised test accuracy
   B. At random
   C. The value that maximised profit on development data
   D. Always 0.5
5. Which safeguard belongs in the board report?
   A. Never retrain the model
   B. A human appeal process and regular retraining
   C. Hide the model from staff
   D. Remove the threshold
6. Why report the effect on good borrowers?
   A. It is not important
   B. Refusing good customers has a real cost to them and to the SACCO
   C. To increase AUC
   D. Because accuracy is 100%

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.